

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work No. 4

Operating Systems

The xv6 Shell from the Outside

Supervisors:
Dumitraşcu Marius, University assistant
Reiţman Patricia, University assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks and results **3**

2.1 Environment and evidence 3

2.2 Part 1 - Fix sleep for the current xv6 3

2.3 Part 2 - Bash versus the xv6 shell 4

2.4 Part 3 - Who expands the arguments? 5

2.5 Part 4 - Read the shell 7

3 Conclusions **9**

A Submitted source code **10**

A.1 user/sleep.c 10

A.2 user/args.c 10

A.3 args_linux.c 10

1 Introduction

Compare Bash with the xv6 shell, inspect the argument vector delivered to a program, and connect observed behavior to the shell implementation.

This report follows the teacher's handout and includes every required Observe answer. Terminal-style listings replace new screenshots. Recorded output is identified with its execution environment; shortened excerpts are marked. Source files and build instructions are provided in the submission directory.

2 Tasks and results

2.1 Environment and evidence

The C programs were built and executed on 23 October 2026. Linux tests used Bash on Debian GNU/Linux, ARM64, as the non-root user `student`. xv6 was compiled with the RISC-V GNU toolchain and booted in QEMU, with one emulated CPU, from MIT xv6-riscv commit `f780d7850765d74cbdcd1fff4b822d8829f7c9bd`. A combined test image contained the lab 04, 06 and 08 programs. Each submitted Makefile was also checked independently. Inodes, PIDs, file sizes and listings depend on the image and run.

The terminal blocks below are recorded output, with trailing prompts removed and long listings shortened only where explicitly marked. `terminal-xv6.txt` and `terminal-linux.txt` contain the complete captured sessions. Interactive history/completion behavior is explained separately; automated tests did not simulate those keypresses.

2.2 Part 1 - Fix sleep for the current xv6

`user/user.h` declares `int pause(int);`. The submitted `user/sleep.c` therefore calls `pause(atoi(argv[1]))`. The command name remains `sleep`; it is the system-call name that changed. The number supplied is in timer ticks, not seconds. The program requires exactly one argument and uses status 1 for a usage error.

Build from the xv6 root after copying the submission overlay:

```
grep -n 'pause\|sleep' user/user.h
make qemu
```

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

The first command returned after its pause. The second printed the required diagnostic. The Makefile includes `sleep` and `args` in `UPROGS`; the new source files belong under `user/`.

2.3 Part 2 - Bash versus the xv6 shell

The Bash comparison begins in an empty directory containing `README`, as required. The argument-expansion experiment later uses its own directory containing `a.c` and `b.c`.

Command or feature	Bash result	xv6 result and explanation
`echo hello > f ; cat < f`	`hello`	`hello`; redirection works and LIST waits for the first command.
`ls | grep READ`	`README`	`README` with its type, inode and size; `ls` uses a different format.
`(echo a; echo b) | wc`	`2 2 4`	`2 2 4`; grouping and a pipeline work.
`echo one ; echo two`	`one`, then `two`	Same sequence; LIST calls `wait(0)`.
`echo bg &`	`bg`; an interactive Bash also displays job information	`bg`; the shell can prompt before it finishes, without a Bash job table.
`ls *`	Expands the names `README` and `f`	`ls: cannot open *`; no wildcard expansion.
`echo $HOME`	`/home/student`	Literal `$HOME`; no variable expansion.
`echo "a b"`	`a b`	`"a b"`; quotes remain ordinary characters.
`mkdir d ; cd d ; ls ..`	Enters `d` and lists `README`, `d`, `f`	`exec cd failed`; the list parser treats `cd` as an external program.
Up, Tab, Ctrl+R	Interactive Readline provides history, completion and reverse search	These Bash facilities are absent; xv6's line input does not implement them.

Both shells support executable commands, redirection, pipes, lists, background execution and parenthesized grouping. Bash adds pathname and variable expansion, quote processing, PATH search, richer built-ins and interactive editing/job management. A noninteractive Bash script normally does not print the interactive job-number notification.

Recorded xv6 checks:

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README         2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
```

```
one
two
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
```

2.3.1 Correction to the partial report

The earlier explanation attributed `open f failed` to xv6 not waiting between semicolon-separated commands. That explanation is incorrect. In the inspected `user/sh.c`, the LIST branch forks the left command, calls `wait(0)`, then runs the right command. The new controlled run above prints `hello`. The earlier failure could have involved the previous disk state or a different local tree; the available evidence does not establish its cause.

2.3.2 Why ls stops working after cd

A standalone `cd d` is handled in the parent shell. Once inside `d`, bare `ls` is passed to `exec` as a relative path. xv6 has no PATH search and its programs live in `/`, so use `/ls` or `../ls`.

```
$ cd d
$ ls
exec ls failed
$ /ls
.              1 34 32
..             1 1 1024
$ ../ls
.              1 34 32
..             1 1 1024
$ cd /
```

The compound command `mkdir d ; cd d ; ls ..` is a separate limitation: the special `cd` handling checks only the beginning of the input line, before parsing it. Within a parsed list, `cd` is treated as a missing executable. A child's `chdir` would not update the shell parent's current directory.

2.3.3 Which feature I would add

I would most miss executable search during everyday use, because changing directory otherwise breaks short command names. A simple fallback that tries `/command` after a relative `exec` failure would be the easiest useful first extension. Full Bash PATH variables, quoting and globbing would require additional parsing and expansion logic.

2.4 Part 3 - Who expands the arguments?

`user/args.c` and `args_linux.c` print `argc` and each `argv` entry. xv6 uses its headers and `exit(0)`; Linux uses `<stdio.h>` and `return 0`. To reproduce the Linux commands with the exact executable name `args`:

```
gcc -Wall -Wextra -Werror -o args args_linux.c
# Use an isolated test directory with a.c and b.c.
```

```
# Add the directory holding args to PATH, or invoke ./args instead.
```

For `args *.c "x y"`, the controlled Linux directory contains exactly `a.c` and `b.c`. Both examples have `argc = 4`, which includes `argv[0]`, but the argument contents differ:

```
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [a.c]
argv[2] = [b.c]
argv[3] = [x y]
```

```
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

Bash supplies three user arguments: `a.c`, `b.c`, and the single string `x y`. xv6 supplies the literal `*.c` and two strings containing the double-quote characters: `"x` and `y"`. Equal argument counts here are a coincidence; a different number of matching C files changes Bash's count.

The shell expands the wildcard and removes syntactic quotes before calling exec. Neither the kernel nor the argument-printing program performs those expansions. xv6's kernel simply receives the argv strings constructed by its shell.

2.4.1 Repeated spaces

```
$ args one   two  three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
```

```
$ args one   two  three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
```

Both shells use runs of unquoted whitespace as separators, so the extra spaces produce no empty arguments. Bash can preserve spaces inside quotes, while the xv6 tokenizer does not recognize quoting.

2.4.2 Variables and single quotes

```
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [/home/student]
$ args 'single quoted'
argc = 2
```

```
argv[0] = [args]
argv[1] = [single quoted]
```

```
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

Bash substitutes HOME and treats a single-quoted string as one literal argument with the quotes removed. xv6 passes `$HOME` literally and splits the single-quoted phrase at its space.

2.5 Part 4 - Read the shell

The commands below refer to the pinned source checkout. Its `user/sh.c` has 499 lines. The values and line numbers are from that checkout; other revisions may differ.

```
$ wc -l user/sh.c
499 user/sh.c
$ grep -n '#define' user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
$ grep -n 'case ' user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
[additional tokenizer/parser cases omitted]
$ grep -n 'cd' user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

2.5.1 Five command kinds and their syntax

Kind	Syntax	What the shell does
EXEC	A command name and its arguments	Calls `exec(argv[0], argv)`.
REDIR	<, >, »	Closes the selected descriptor, opens a file and runs its child command.
PIPE	|	Creates a pipe and two child processes, connects stdout to stdin, waits for both.

Kind	Syntax	What the shell does
LIST	`;`	Runs the left command, waits, then runs the right command.
BACK	`&`	Forks the command and allows the supervising shell to continue.

Parentheses group a parsed command; they are not a sixth command kind. In this xv6 version, » opens without truncation but does not implement a true O_APPEND seek-to-end operation; the required experiment uses >.

2.5.2 Where cd is handled

`main()` checks `cmd[0]`, `cmd[1]` and `cmd[2]` for `cd` at line 166 and calls `chdir(cmd + 3)` at line 169, before forking a command. This must happen in the shell parent because the working directory is per process.

2.5.3 Searching for expansion

```
grep -nF '*' user/sh.c
grep -nF '$' user/sh.c
sed -n '270,320p' user/sh.c
```

A literal search for `*` finds pointer declarations and dereferences, not pathname-expansion code. `$` occurs in the prompt string. The tokenizer treats words as sequences bounded by whitespace and the syntax symbols `<|>&;()`. There is no stage that resolves wildcards, substitutes environment variables or strips quotes. This explains the literal `*`, `$HOME`, and quote characters in Parts 2 and 3.

3 Conclusions

The experiments distinguish shell syntax from kernel mechanisms. xv6 supplies process execution, lists, pipes and redirection, while Bash adds expansion, quoting, executable search and interactive facilities. Printing argv makes these differences explicit. Source inspection also corrects the earlier claim about semicolon ordering: the LIST branch waits before executing its right-hand command.

Bibliography

[1] Operating Systems, FCIM / FAF, Technical University of Moldova. Laboratory work No. 4: *The xv6 Shell from the Outside*. Course handout, 2026–2027.

[2] MIT PDOS. *xv6-riscv*, commit `f780d7850765d74cbdcd1fff4b822d8829f7c9bd`. Source checked on 23 October 2026. `https://github.com/mit-pdos/xv6-riscv/tree/f780d7850765d74cbdcd1fff4b822d8829f7c9bd`.

[3] Linux man-pages project. *fork(2), pipe(7), inode(7), path_resolution(7)*. `https://man7.org/linux/man-pages/`.

A Submitted source code

The following listings read the actual submitted source files, so the report and submission use the same implementation. Variant programs are included where relevant.

A.1 user/sleep.c

```c
#include "kernel/types.h"
#include "user/user.h"
int main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }
    pause(atoi(argv[1])); // system call into the kernel
    exit(0);
}
```

A.2 user/args.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

A.3 args_linux.c

```c
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```